CN SDN Mini Project

Bandwidth Measurment and Analysis

NIKHIL B MENON

PES2UG24CS317

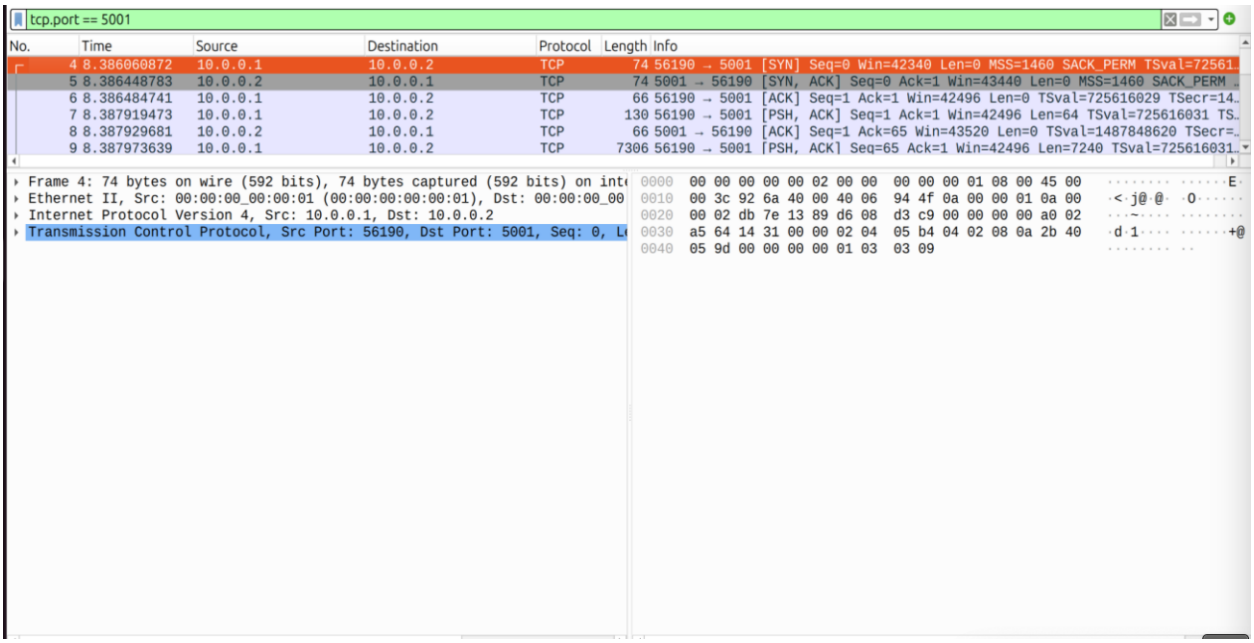
Github-Link: https://github.com/nikhilbipinmenon/CN-miniproject/

Abstract

This project demonstrates Software Defined Networking (SDN) using Mininet and a POX controller to analyze and control network bandwidth. A simple topology is used to study controller–switch interaction and OpenFlow-based flow rules. Network performance is evaluated using ping and iperf under normal and blocked conditions. The results show how SDN enables dynamic control of traffic and network behavior. Overall, the project highlights the flexibility of centralized network management in SDN.

Scenario 1:

```
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> h2 iperf -s &
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
mininet> h1 iperf -c 10.0.0.2 -t 5
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 46948 connected with 10.0.0.2 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-5.0106 sec  23.0 GBytes  39.5 Gbits/sec
mininet> h1 iperf -c 10.0.0.2 -t 5
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 56190 connected with 10.0.0.2 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-5.0268 sec  5.96 GBytes  10.2 Gbits/sec
mininet>
```



Flow rules being added:

```
nikhil@nikhil:~$ cd pox
nikhil@nikhil:~/pox$ python3 pox.py controller
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.13.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:controller:Controller connected to switch
INFO:controller:Learned 00:00:00:00:00:02 on port 2
INFO:controller:Learned 00:00:00:00:00:02 on port 2
INFO:controller:Learned 00:00:00:00:00:01 on port 1
INFO:controller:Learned 00:00:00:00:00:02 on port 2
```

Scenario 2:

```
mininet> sh ovs-ofctl del-flows s1
mininet> sh ovs-ofctl add-flow s1 priority=100,actions=drop
mininet> pingall
*** Ping: testing ping reachability
h1 -> X
h2 -> X
*** Results: 100% dropped (0/2 received)
mininet> h1 iperf -c 10.0.0.2 -t 5
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] tcp connect to 10.0.0.2 port 5001 failed (No route to host) on 2026-04-23 09:49:02 (IST)
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
mininet>
```

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	00:00:00_00:00:01	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
2	1.026388122	00:00:00_00:00:01	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
3	2.050371276	00:00:00_00:00:01	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
4	19.599515714	00:00:00_00:00:01	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
5	20.610344955	00:00:00_00:00:01	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
6	21.634439648	00:00:00_00:00:01	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1

```
▸ Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on inte
▸ Ethernet II, Src: 00:00:00_00:00:01 (00:00:00:00:00:01), Dst: Broadcast (
▸ Address Resolution Protocol (request)

0000  ff ff ff ff ff ff 00 00  00 00 00 01 08 06 00 01   ........ ........
0010  08 00 06 04 00 01 00 00  00 00 00 01 0a 00 00 01   ........ ........
0020  00 00 00 00 00 00 0a 00  00 02                     ........ ..
```

```
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=272.577s, table=0, n_packets=17, n_bytes=826, priority=100 actions=drop
mininet>
```

ARP requests are sent but dropped by the switch due to the drop rule, preventing ARP resolution.